

Scalable Parking Lot

Multi-floor OOP design, nearest-spot assignment, HashMaps and heaps

A classic object-oriented design interview. They want vehicle polymorphism, $O(1)$ lookups, a clear park/unpark flow, and a story for adding floors, spot types, and pricing without rewriting the lot.

How to use this document in an interview: start with the 90-second pitch, draw the class diagram, name the data structures and their complexities, then walk the core flow. Use the SOLID and follow-up sections when the interviewer pushes on extensibility.

1. 90-second pitch

A ParkingLot is a composition of floors, each floor a list of ParkingSpots. Vehicles are a small hierarchy -- Bike, Car, Truck -- that advertise a required spot size. A bike may take a larger stall; a truck may not take a smaller one. On park I pick the nearest compatible stall: lowest floor, then lowest spot number. I do not scan the garage. I keep a min-heap of free stall ids per size, plus HashMaps from plate to ticket and ticket id to ticket. Park and unpark are $O(\log n)$ for the heap and $O(1)$ for the maps. Unpark closes the Ticket, which owns fee calculation, and pushes the stall back onto the heap.

SAY THIS: Lead with data structures. 'I would put vehicles in a list and for-loop for a free spot' is the junior answer. HashMap + heap is the expected upgrade.

2. Problem statement and requirements

Functional

- Multiple floors, each with a configurable mix of SMALL / MEDIUM / LARGE spots.
- Vehicle types: bike, car, truck -- different minimum sizes.
- park_vehicle(vehicle) -> Ticket, assigning the nearest compatible spot.
- unpark_vehicle(ticket_id) -> closed Ticket with fee.
- get_available_spots() -- free counts by floor and size.
- Reject a plate that is already parked. Reject a full-compatible-lot.

Non-functional

- Park / unpark / 'is this plate inside?' must be fast as the garage grows.
- Adding a floor or a new vehicle type should not rewrite ParkingLot.park_vehicle.
- Clear domain exceptions, not boolean flags, for full / duplicate / unknown ticket.
- Fee policy isolated so hourly vs daily vs EV surcharge can change.

Assumptions to say first

- Nearest = (floor ascending, spot number ascending). Spot 1 on a floor is closest to the entrance.
- A vehicle may use a larger stall if no exact size is free and that larger stall is still nearer.
- One vehicle per plate. One open ticket per vehicle.
- Minimum one hour billed, then ceil to the next hour. Bike \$1, car \$3, truck \$6.
- Single-threaded unless they ask. I will mention locking as an extension.
- No reservations, no EV chargers, no handicap-only spots -- until they add them.

3. High-level design

Class diagram

```

Vehicle <--|-- Bike      required_size = SMALL
      +-- Car      required_size = MEDIUM
      +-- Truck    required_size = LARGE
      |
      | parks in
      v
ParkingSpot (floor, number, size, vehicle?)
      ^
      | 1..*
ParkingFloor
      ^
      | 1..*
ParkingLot <<facade>>
      park_vehicle / unpark_vehicle
      get_available_spots
      |
      | issues
      v
Ticket (id, vehicle, spot, in, out, fee)

create_vehicle(type, plate) <<factory>>

```

Module map in this repo

Topic	What to say / what we did
parking/types.py	VehicleType, SpotSize, compatibility table, hourly rates.
parking/vehicle.py	Vehicle hierarchy + create_vehicle factory.
parking/spot.py	One stall. Occupancy and can_fit.
parking/floor.py	Builds numbered spots. Counts free stalls.
parking/ticket.py	Entry/exit timestamps and fee.
parking/lot.py	Indexes, nearest assignment, park/unpark.

4. Class responsibilities

Vehicle / Bike / Car / Truck

Identity is the license plate (normalized to uppercase). Subclasses do not override behavior; they declare class-level `vehicle_type` and `required_size`. That is polymorphism by data: `ParkingLot` never writes `if isinstance(v, Truck)`. It asks `v.required_size`. Adding `Van(required_size=MEDIUM)` is a new subclass plus one factory map entry.

SpotSize and compatibility

`SMALL=1`, `MEDIUM=2`, `LARGE=3`. A spot can fit a vehicle when `spot.size >= required`. `COMPATIBLE_SIZES` is the inverted index the heap walker uses:

```
SMALL -> SMALL, MEDIUM, LARGE
MEDIUM -> MEDIUM, LARGE
LARGE -> LARGE
```

ParkingSpot

Knows its coordinates, size, and current vehicle. `park/unpark/can_fit` live here so the lot cannot put a truck in a small stall even if a bug skipped the heap filter -- defense in depth.

ParkingFloor

Constructs spots in entrance order: all `SMALL`, then `MEDIUM`, then `LARGE`, numbered from 1. It does not assign vehicles. That keeps 'where stalls live' separate from 'which stall is chosen' (SRP).

Ticket

Created at park, closed at unpark. Fee is computed only at `close()` so an open ticket does not hold a stale price if rates change mid-stay (rates are read at close). Minimum one hour, then ceiling.

ParkingLot

The facade. It owns floors, the HashMaps, and the heaps. External code should not reach into spots to park. This is also the transaction boundary: park updates spot + ticket map + plate map together.

5. Data structures (the core of the interview)

This is where you outrun a linear scan. Five indexes, each justified:

Topic	What to say / what we did
<code>_spots: dict[spot_id, Spot]</code>	$O(1)$ resolve a heap entry or a ticket's stall.
<code>_active_tickets: dict[id, Ticket]</code>	$O(1)$ unpark by ticket id.
<code>_parked: dict[plate, Ticket]</code>	$O(1)$ duplicate-plate check and plate lookup.
<code>_available: dict[size, min-heap]</code>	Nearest free stall of a given size, $O(\log n)$ push/pop.
<code>_available_ids: dict[size, set]</code>	Membership so stale heap entries can be discarded.

Why a heap, not a scan

Naive park: for floor in floors: for spot in floor.spots: if spot.can_fit(v): take it. That is $O(S)$ per park, S = total stalls. Fine for 50 spots, poor for a stadium. A min-heap keyed by (floor, number, id) gives the nearest free stall of one size in $O(1)$ peek and $O(\log n)$ pop.

Nearest across compatible sizes

A bike can sit in three heaps. I peek the live head of each compatible heap and take the min (floor, number). Then I mark that id removed. I do not pop the other heaps. Example: floor 1 smalls are gone, floor 1 medium #4 vs floor 2 small #1 -- (1,4) wins, so the bike takes a medium on floor 1. That matches 'nearest', not 'prefer exact size even if farther'.

Lazy heap deletion

heapq cannot delete an arbitrary element in $O(\log n)$ without extras. On assign I remove the id from `_available_ids` and leave the tuple in the heap. The next peek pops stale heads until it finds an id that is still live. Amortized cheap if we do not churn the same stall pathologically.

WATCH OUT: If they say 'I need decrease-key / delete', offer a position map + tombstones, or a balanced tree (sorted set) of (floor, number, id). Python's heapq is the honest stdlib choice.

Why HashMap for plates

Duplicate parking is a real bug (two tickets, one car). A linear search of active tickets is $O(\text{parked})$. A dict is $O(1)$ and is the sentence they want: 'HashMap for quick lookup of parked vehicles.'

6. Key flows

park_vehicle

1. If plate in `_parked`: raise `VehicleAlreadyParkedError` (include existing spot).
2. `spot = _pop_nearest(vehicle)`. If None: raise `NoSpotAvailableError`.
3. `spot.park(vehicle)` -- second check on size and occupancy.
4. Create Ticket. Index by `ticket_id` and plate. Return ticket.

unpark_vehicle

1. Pop ticket from `_active_tickets` or raise `UnknownTicketError`.
2. `spot.unpark()`. `ticket.close()` -- writes exit time and fee.
3. Remove plate from `_parked`. `_push_available(spot)`.
4. Return the closed ticket so the booth can print the fee.

get_available_spots

Walks floors and counts free stalls by size. $O(S)$. This is a report, not the hot path. If a dashboard needed $O(1)$, I would keep running counters updated on push/pop -- mention that as a follow-up optimization.

7. OOP principles

Encapsulation

Occupancy is `ParkingSpot._vehicle`. Availability indexes are private on the lot. The console uses `park_vehicle` / `unpark_vehicle` / `get_available_spots` only.

Abstraction

Callers do not know there is a heap. They know 'nearest compatible spot'. That lets us swap the heap for a tree later without touching `parking_main`.

Inheritance (where it earns its keep)

Vehicle is the only hierarchy. Subclasses are substitutable: any Vehicle can be parked. We do not inherit `ParkingSpot` into `CompactSpot` / `HandicapSpot` unless behavior diverges -- a size enum plus a flags set is enough for most variants.

Polymorphism

`create_vehicle` returns `Vehicle`. `park_vehicle` accepts `Vehicle`. Fee uses `vehicle.vehicle_type` to index `HOURLY_RATE`. New types plug in without editing the loop in `park_vehicle`.

Composition

Lot has floors; floors have spots; tickets have a vehicle and a spot. The garage is a tree of has-a, not a God class with 40 fields.

8. SOLID principles

S -- Single Responsibility

Topic	What to say / what we did
Vehicle	Who is parking. Not fees, not heaps.
ParkingSpot	Can this stall hold this vehicle right now?
ParkingFloor	Layout of one level.
Ticket	Stay duration and money.
ParkingLot	Assignment policy + indexes + use-case API.
parking_main.py	Human I/O.

Honest gap: `HOURLY_RATE` lives in `types.py`. A `PricingPolicy` class would be a cleaner SRP split if pricing rules grow (grace period, night rate, EV). I would say that unprompted -- it scores more than pretending the design is finished.

O -- Open/Closed

Closed: `park_vehicle` does not change when we add Van or a third floor. Open: new `Vehicle` subclass, new factory entry, optional new `SpotSize`. `PricingPolicy` would make rate changes closed as well.

If they add EV-only stalls: either a new `SpotSize`/flag and a filter on the heap key, or a separate heap. Do not put if `vehicle.is_ev` inside five classes -- extend the compatibility table.

L -- Liskov Substitution

Any Vehicle passed to `park_vehicle` must be parkable using `required_size`. A subclass that leaves `required_size` unset, or that throws in `__str__`, would violate LSP. Bike/Car/Truck only specialize two class attributes -- `safe`.

I -- Interface Segregation

The lot's public API is three verbs plus lookups. Floors do not implement `park`. Spots do not implement `fee`. Clients are not forced to depend on heap internals.

D -- Dependency Inversion

ParkingLot depends on the Vehicle abstraction, not on Car. The factory returns Vehicle. Ticket.close depends on a rate table keyed by VehicleType -- if we inject a PricingPolicy, Ticket depends on an interface, not a dict in `types.py`. That is the DIP improvement I would make next.

9. Design patterns

Topic	What to say / what we did
Factory Method / Simple Factory	<code>create_vehicle(type, plate)</code> hides Bike/Car/Truck constructors from the UI.
Facade	ParkingLot is the one object the application talks to.
Composite-style tree	Lot -> Floor -> Spot. Operations roll up (available counts).
Value / entity split	Ticket and Spot are entities (identity). Vehicle is an entity keyed by plate. SpotSize is a value.
Strategy (ready)	Nearest-spot policy and pricing are the two strategies I would extract if a second policy appears (e.g. 'prefer exact size').
Template-ish flow	<code>park</code> is a fixed sequence: <code>validate</code> , <code>allocate</code> , <code>occupy</code> , <code>ticket</code> . Hooks would be <code>allocate()</code> and <code>price()</code> .

Not used, and I would say so: Singleton (one global lot kills tests and multi-garage apps), Observer (gate displays can poll `get_available_spots`), Object Pool (spots are not disposable workers).

10. Complexity cheat sheet

Topic	What to say / what we did
park_vehicle	$O(1)$ plate check + $O(k \log n)$ nearest, k = compatible sizes (max 3) ~ $O(\log n)$.
unpark_vehicle	$O(1)$ map ops + $O(\log n)$ heap push.
find_vehicle / find_ticket	$O(1)$.
get_available_spots	$O(S)$ scan, S = total spots. Optional: $O(F)$ with counters.
Memory	$O(S)$ spots + $O(S)$ heap entries + $O(P)$ tickets, P = parked cars.

11. Concurrency and scale (they will ask)

This implementation is single-threaded. Two booths calling `park_vehicle` at once could pop the same heap head. Fixes, in order of complexity:

- A single `asyncio/thread` lock around `park` and `unpark`. Simple, correct, may serialize booths.
- Per-size locks so a bike heap and a truck heap can proceed in parallel.
- Allocate in a DB transaction with `SELECT FOR UPDATE` on the chosen spot row -- the production version.
- Distributed lot: Redis heap or a parking service with a lease (spot reserved for 5s while payment confirms).

Horizontal scale: the in-memory maps become Redis hashes; the heap becomes a sorted set (ZADD score = floor*K + number). The class diagram stays the same. That is the line that makes the design sound scalable rather than 'I used a dict'.

12. Extensions interviewers love

- Handicap / EV / compact: add flags on ParkingSpot; filter in `_pop_nearest`.
- Reservations: hold a spot with an expiry; a sweeper returns expired holds to the heap.
- Dynamic pricing: PricingPolicy(time, occupancy_ratio, vehicle_type).
- Multi-entrance nearest: distance is not (floor, number) but Euclidean to the entry used. Heap key becomes (distance, id).
- Payment: Ticket stays; a PaymentService charges on close. Do not put Stripe in ParkingLot.
- Display boards: Observer or a scheduled `get_available_spots`.

13. Follow-up questions (with answers)

Bike vs a closer large spot vs a farther small?

I defined nearest as physical (floor, number) among compatible stalls. A farther exact-size stall loses. If the product wants 'never waste a large on a bike if a small exists anywhere', that is a different strategy: walk compatible sizes in size order, not by distance. I would ask which rule they want.

What if the lot has 10,000 spots?

Heaps stay $O(\log n)$. Maps stay $O(1)$. `get_available_spots` should switch to counters. Memory is the real limit in-process -- at that size I persist spots and keep only free-id heaps hot.

How do you prevent double parking the same plate?

`_parked[plate]` checked before allocation. Plate is normalized. I would also unique that column in a database.

Where does money live?

`Ticket.close()`. ParkingLot is not a cashier. If receipts need tax, a PricingPolicy or BillingService is the next class -- not more if-else in the lot.

Why not one list of free spots sorted by distance?

Then every park must filter by size while scanning from the front -- worst case $O(S)$. Per-size heaps jump to a compatible candidate in $O(\log n)$.

How would you unit-test nearest?

Build a tiny lot: 2 small, 2 medium, 1 large on two floors. Park a bike, assert F01-S001. Park a car, assert first medium. Fill smalls, park another bike, assert it took the nearest medium, not a small on floor 2 if floor 1 medium is closer. That test is in the design we shipped.

14. Whiteboard script (15 minutes)

1. Clarify: floors, sizes, nearest definition, leftover large-for-bike rule, fees.
2. Draw Vehicle hierarchy and Spot / Floor / Lot / Ticket.

3. Write the three public methods and the exception cases.
4. Draw the five indexes. Circle HashMap and Heap -- say complexities.
5. Walk park then unpark with a bike, car, truck example.
6. SOLID in 60 seconds, then name Factory + Facade.
7. Extensions: EV, lock, Redis sorted set, PricingPolicy.

15. Common mistakes to avoid

- One ParkingLot class that also computes fees, prints tickets, and talks to payment.
- instanceof(vehicle, Car) trees -- breaks OCP.
- Scanning all spots every park and calling it 'scalable' because 'computers are fast'.
- Forgetting to return a freed spot to the free index (leak of capacity).
- Allowing the same plate twice.
- Putting console input() inside ParkingLot.
- Calling the lot a Singleton so 'there is only one garage in the world'.

16. Side-by-side with the cards problem

Same interview muscle: domain types, one facade, private data structures, fail-fast errors, UI kept out. Difference: cards is an algorithm problem (prove Fisher-Yates). Parking is an indexing problem (prove you would not scan). If you get both in one loop, say that sentence -- it shows you can classify the problem, not just dump patterns.